Module 4: Semantic Analysis and Intermediate Code Generation

Question 7 (3 marks each):

- **A-7**: Differentiate between synthesized attributes and inherited attributes with an example.
- **B-7**: Explain quadruples, triples, and indirect triples with suitable examples.
- **D-7**: What are the advantages of indirect triples compared to triples?
- **E-7**: Explain the structure of an activation record.
- **F-7**: Draw the syntax tree and DAG representation for a given statement.
- **July 2023-7**: Convert the expression `a = b * -c + b * -c` into quadruples.

Question 8 (3 marks each):

- **A-8**: What is the role of an activation record in compiler design?
- **B-8**: What are L-attributed definitions and S-attributed definitions in a syntax-directed translation scheme?
- **D-8**: Write three-address code for a while loop by choosing a suitable example.
- **E-8**: Compare L-attributed and S-attributed syntax-directed definitions.
- **F-8**: What are L-attributed definitions and S-attributed definitions in a syntax-directed translation scheme?
- **July 2023-8**: Define SDD with an example.

Question 17 (14 marks each):

- **A-17**:
  a) Write the SDD for a simple type declaration and draw the annotated parse tree for the declaration `float a, b, c`.
  b) With an SDD for a desk calculator, write the steps involved in the bottom-up evaluation for the expression `(3 * 5) - 2`.
- **B-17**:
  a) Write the SDD for a desk calculator and draw the annotated parse tree for the expression: `4 * 5 + 6 - (3 * 2)`.
  b) Explain bottom-up evaluation of S-attributed definitions.
- **D-17**:
  a) Write a syntax-directed definition for a simple type declaration involving basic types such as `int`, `float`, and `char` (assume C programming language syntax).
  b) Write an SDD for generating a syntax tree for arithmetic operations.
- **E-17**:
  a) With the help of a syntax-directed definition of a simple desk calculator, evaluate the expression `(3 + 5 / 2) * (2 + 4 / 3)` and draw the annotated parse tree.
  b) Write an SDD for a simple type declaration. With the SDD, write the steps involved in the evaluation of inherited attributes for the statement `int a, b, c`.
- **F-17**:
  a) Write the syntax-directed translation scheme for a simple desk calculator. Give an annotated parse tree for the input string `23 * 5 + 4`.
  b) Explain static allocation and heap allocation strategies.
- **July 2023-17**:
  a) Define the following terms with suitable examples: (i) Three-address code, (ii) Triples, (iii) Quadruples.
  b) Explain static allocation and heap allocation strategies.

Question 18 (14 marks each):

- **A-18**:
  a) Explain static allocation and heap allocation strategies.
  b) Construct the DAG and three-address code for the expression `a + a * (b - c) + b * (b - c) + b`.
- **B-18**:
  a) Write a syntax-directed definition to construct a syntax tree and three-address code for assignment statements.
  b) Explain static allocation and heap allocation strategies.
- **D-18**:
  a) Draw the DAG representation for the statement:
  `s = (a + b) * (b + c) + (a + b)`.
  b) Construct quadruple, triple, and indirect triple tables for the above DAG representation.
- **E-18**:
  a) Write a note on stack allocation, heap allocation, and static allocation strategies.
  b) What are the different representations of three-address code? Write the different three-address code representations for the expression:
  `(a + b) * (b + c) * (a + b + c)`.
- **F-18**:
  a) Write the quadruple, triple, and indirect triple for the expression:
  `(a * b) + (c + d) - (a + b + c + d)`.
  b) Generate the intermediate code for the code segment:

```c
if (a > b)
    x = a + b
else
    x = a - b
```

- **July 2023-18**:
  a) Construct a syntax-directed definition for the assignment statement:
  `S → E`, `E → E1 + E2`, `E → E1 * E2`, `E → -E1`, `E → (E1)`, `E → id`.
  Also, construct an annotated parse tree for the input string `6 * 8 + 5`.
  b) Generate intermediate code for the code segment:

```c
if (a > b)
    x = a + b
else
    x = a - b
```

where `a` and `x` are of real type and `b` is of int type, along with the required syntax-directed translation scheme.